

FULL STACK DEVELOPMENT

FASE 5 | AULA 01 -

A CLEAN ARCHITECTURE

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	10
REFERÊNCIAS.....	11

EMSE

O QUE VEM POR AÍ?

Entender como construir uma arquitetura de software e de soluções é um dos requisitos para a entrega de um software de qualidade. Entender as diferentes abordagens para a construção de uma arquitetura, considerando seus pontos positivos e negativos, permite nos basear em ideias já testadas e validadas.

Uma das abordagens mais utilizadas para isso é a Clean Architecture (ou Arquitetura Limpa). Mais do que um guia de uso, essa abordagem nos mostra como escrever códigos, como estruturar os componentes e como resolver problemas comuns no ciclo de vida do software.

HANDS ON

Aprenderemos bastante teoria no “saiba mais”, mas agora a discussão será mais prática, com algumas comparações com elementos da nossa vida cotidiana. Também vamos discutir um pouco sobre alguns modos de implementação de SOLID e porque código limpo é importante para o sucesso de um projeto de longo prazo.



SAIBA MAIS

Arquitetura de software é um campo amplo de estudos, que se baseia parte em estudo e parte em análise de casos. Isso porque existem inúmeras ideias de arquitetura de software e, como a disciplina de criação de software é algo novo, ainda não temos as principais práticas solidificadas. Isso não é algo ruim, é uma oportunidade de aprender cada vez mais com as possibilidades de resolver o mesmo problema de formas diferentes, com abordagens diferentes e até mesmo com objetivos distintos, como construir algo que funcione com maior eficiência de custo operacional ou para rodar em diferentes infraestruturas ou dispositivos. Em programação, criar um software e criar soluções muitas vezes acaba sendo a mesma coisa.

Criar um tipo de arquitetura que permita que o software atinja seus objetivos de forma eficiente e eficaz, e que ainda seja simples de alterar e de entender, é um desafio que surge junto com a programação, e aumenta de complexidade conforme vamos criando softwares mais complexos e bases de código cada vez maiores. E quanto maior a base de código, melhor deve ser a estrutura desse código, melhores devem ser os testes e definições de regras de negócio e melhores devem ser os processos que garantam que o software funcione desde a implementação de melhorias até a disponibilização para os usuários. Um software complexo precisa de uma estrutura robusta à sua volta para que funcione bem.

Arquitetura limpa é um conceito amplo e desafiador de definir, porque não está vinculada a uma verificação mensurável em um tipo de métrica, ou com algum “atestado de qualidade”. A arquitetura limpa, como definida no livro de Robert C. Martin, está relacionada à qualidade da construção de software e à criação de um código estruturado e fácil de entender e alterar. Ainda que seja difícil de definir, é fácil identificar um código com arquitetura limpa, ou um código que não tem essa qualidade, porque quanto mais experiências temos com o desenvolvimento de software e de arquiteturas, mais entendemos os problemas de uma arquitetura ruim e a necessidade de uma arquitetura bem definida.

O que Robert Martin faz desde o primeiro livro de sua série “Clean Code”, é definir boas práticas, métodos e ideias de como construir software com algumas

qualidades bem definidas: fácil de manter, fácil de entender, com limites bem definidos em suas camadas, e que usa características das linguagens em que o código é implementado para garantir as qualidades citadas anteriormente; um exemplo claro é usar bastante alguns conceitos de orientação a objeto, como interfaces e herança, para garantir que o código funcione de forma organizada e, ao mesmo tempo, com pouco acoplamento.

Os primeiros livros, “Clean Code” (Código Limpo) e “Clean Coder” (Codificador Limpo) discorrem sobre a construção de um código limpo, fácil de manter e entender, e sobre como ter a mentalidade para tomar decisões que fazem o código se manter bem cuidado, bem-feito e bem construído.

O uso da Arquitetura Limpa nos ajuda a aplicar conceitos importantes para manter o código funcionando bem, corretamente e com simples de manutenção. Sem este tipo de organização, a implantação de metodologias ágeis se torna complicada, porque não é possível aplicar a agilidade em códigos difíceis de manter. A Arquitetura Limpa, quando define ideias de forma clara, bem como as regras para uso na programação e na construção do software, ajuda a garantir que o código vai funcionar como esperado.

O código criado com base na Arquitetura Limpa não é mais conciso, com menos classes ou com técnicas de programação complicadas. Ao contrário, ele se baseia na estruturação do código com base em algumas regras de codificação e arquitetura, e nem sempre a estruturação do código vai gerar um código menor. Aliás, isso não acontece, mas um dos princípios que definem a boa arquitetura de software é o de que tudo esteja sempre explícito, sem regras que causem dúvida no momento da implementação ou manutenção.

É importante que o código seja legível e bem estruturado para garantir que o código seja alterado, permitindo corrigir problemas existentes ou implementar novas funcionalidades. Como pessoas que trabalham mantendo os softwares funcionando, vamos ler muito mais códigos do que escrevê-los. A legibilidade do código nos ajuda a entender as regras de negócio implementadas, os casos de uso que o software resolve, como cada ponto desses é endereçado, e em que pedaço do código reside cada decisão. Um código que está num único bloco é mais complexo de entender do que um distribuído em diferentes componentes, mas se estes componentes não

estiverem bem estruturados, a falta de legibilidade pode ser maior, com os erros e problemas distribuídos entre diversos pedaços de código.

Na aplicação da arquitetura limpa, nós organizamos nossas unidades de código em componentes e precisamos sempre pensar nas unidades como partes de uma solução. Os diferentes componentes são organizados em camadas, e cada camada tem sua responsabilidade.

Pensando em componentes, podemos também isolar as responsabilidades de cada unidade de código, e pensar em unidades menores e especializadas. Isso é uma das regras da codificação de unidades de código, quando trabalhamos com a construção dos componentes. Precisamos de unidades especializadas e com responsabilidades únicas e bem definidas. Não podemos criar uma estrutura com classes generalistas, ou com muito código complexo num mesmo lugar, sob pena de impactar a qualidade do código.

O uso de camadas de software não é uma ideia nova, mas ainda assim é pouco aplicada quando pensamos em código, porque não conseguimos enxergar, de forma visual, o que cada camada faz. Por isso, um outro modo de pensar em camadas são grupos de componentes com responsabilidades semelhantes. No caso de uma camada de Casos de Uso, por exemplo, não podemos ter nenhum código que execute tarefas de conexão com Bancos de Dados, como criar a conexão. Pensando dessa forma, olhamos nosso projeto não como camadas de um bolo, e sim como camadas de uma cebola. É dessa forma que a Arquitetura Limpa pensa a criação das camadas e do código de cada camada.

Usamos, para guiar a qualidade do código, os conceitos de SOLID, definidos nos livros Clean Code e Clean Architecture, de Robert C. Martin. Estes conceitos se tornaram um guia de implementação de código de qualidade e são amplamente discutidos em livros, vídeos e artigos, com aplicação em diferentes linguagens de programação.

SOLID é um acrônimo, em língua inglesa, para cinco princípios:

- **Single Responsibility Principle**, ou **Princípio da Responsabilidade Única**, define que uma unidade de código só deve ter uma responsabilidade e deve ter um único motivo para mudar. Assim, uma unidade de código deve ser especializada no que faz e no uso que é feito dela. Não pode ser

responsável, por exemplo, por alterar sua resposta dependendo do tipo de chamada.

Vamos imaginar uma pessoa atendente de uma lanchonete. Ela tem, no processo de venda de lanches, um propósito específico, que é anotar o pedido, entregar para a cozinha, registrar o pagamento e posteriormente entregar o lanche. A responsabilidade dessa pessoa é clara: servir como uma interface entre o(a) cliente e a cozinha, e faz isso de forma padronizada para todas as requisições que chegam, ou seja, de acordo com os(as) clientes que chegam a cada momento. O código deve ser assim, com uma responsabilidade clara, e que faça apenas o que precisa ser feito.

- **Open-closed Principle** ou **Princípio Aberto-fechado**, define que as unidades de código devem permitir extensão, mas não alteração. Quando precisamos implementar uma nova funcionalidade, não devemos alterar algo que já existe, e sim implementar algo que estenda a funcionalidade de algo existente. Este tipo de ideia nos permite estruturar o código de forma a especializar algo que já existe, sem alterar a implementação atual. Se você precisa de uma funcionalidade que entregue um relatório num formato diferente do que já existe, como uma nova opção, para seguir este princípio é ideal que você pense como pode segregar o código que gera os dados do relatório e, posteriormente, passa os dados para os tipos específicos de um relatório a ser gerado, em vez de abrir o código e colocar mais uma condição de acordo com um novo parâmetro na função já existente.
- **Liskov Substitution Principle** ou **Princípio de Substituição de Liskov**, define que uma classe pode ser substituída por outra do tipo base sem problemas para o software. Nesse ponto, conseguimos garantir que a implementação de uma unidade de código que deriva de outra não viola as regras de uso da classe base, e com isso garantimos que as chamadas sempre funcionam como esperado. O melhor jeito de implementar este princípio, quando estamos usando classes, é com o uso de interfaces. Como veremos posteriormente, quando falarmos de inversão de dependência, a ideia de poder substituir uma classe derivada pela sua classe base permite que o código que usa esta classe não dependa de métodos que só existem em uma das classes filhas.

- **Interface Segregation Principle** ou **Princípio de Segregação de Interface**, define que uma classe não deve implementar interfaces e métodos que não implementará. Seguir esse princípio nos ajuda a criar interfaces mais enxutas e focadas, e a criar implementações que atendem a apenas uma necessidade específica. Aqui vamos sempre pensar nos parâmetros ou componentes que usamos de acordo com o seu comportamento (definido pelos campos e métodos da interface) e não de acordo com sua implementação. Parece algo incomum, mas usamos isso o tempo todo ao ligar algo na tomada: plugamos o dispositivo na tomada, e como a tomada e o dispositivo tem a mesma interface, não importa para o dispositivo como está feita a instalação elétrica, nem onde ele está sendo ligado, só importa se a quantidade de tensão chega corretamente para que o dispositivo funcione.
- **Dependency Inversion Principle** ou **Princípio da Inversão de Dependência**, define que um módulo de alto nível não deve depender de um módulo de baixo nível, sendo que os níveis aqui se referem sempre aos módulos que estão mais perto da infraestrutura. Outro modo de pensar é que uma abstração não deve depender de detalhes de implementação. Por exemplo, um código que precisa de uma origem de dados para efetuar um cálculo não deve depender de uma implementação única dessa origem de dados. Para isso, sempre precisamos que as unidades de código que implementam algo dependam de uma abstração (como uma interface) para efetuar o que é necessário. Isso é vital para que a desconexão entre as diferentes unidades de código aconteça como o esperado. Não podemos aqui confundir **inversão de dependência** com **injeção de dependência**. A inversão de dependência significa passar para o usuário o que ele vai usar sem que ele se importe como foi criado, porque para ele apenas importa fazer funcionar; e um dos **métodos** que vamos usar é **injetar** esse componente para o uso do código.

O QUE VOCÊ VIU NESTA AULA?

Arquitetura simples não é certificação nem padrão de projeto, e sim um conjunto de ideias e recomendações que nos permitem escrever códigos de melhor qualidade, mais fáceis de manter e com a quantidade adequada de unidades de código. Gera mais códigos porque precisamos criar uma quantidade de unidades de código que nos permitam isolar responsabilidades e coordenar o trabalho dessas unidades de códigos de forma simples.

Vimos o SOLID como um conjunto de premissas que serão usadas para as próximas aulas, e entendemos como as unidades de código devem seguir as regras do SOLID para que a Arquitetura Limpa seja implementada com sucesso.

Nas próximas aulas vamos aprender mais regras, focadas principalmente em organizar as unidades de código. O SOLID nos indica como cada unidade deve ser construída e a Arquitetura Limpa indica como organizar e pensar na responsabilidade e na interação entre essas unidades.

REFERÊNCIAS

CLEAN Code e Solid. 2021. Disponível em: <https://www.youtube.com/watch?v=XV9B4LX_re8>. Acesso em: 23 mai. 2023.

MARTIN, R. **Arquitetura Limpa**. Rio de Janeiro: Alta Books Editora, 2019.

MARTIN, R. **Clean Code: A Handbook of Agile Software Craftsmanship**. [s.l.]: Pearson, 2008.

MARTIN, R. **The Clean Coder: A Code of Conduct for Professional Programmers**. [s.l.]: Pearson, 2011.

PALAVRAS-CHAVE

SOLID. Arquitetura Limpa. Camadas de Software.

EMSE

POSTECH